extdrift

Catching malicious browser-extension updates by what they *do*, not what they ask for.

Demonstration brief · CSD493 · prepared for the mid-semester review

An extension you already trust can turn malicious in a routine auto-update — the Cyberhaven incident (December 2024) is the textbook case. Static permission review does not catch this: the update often asks for nothing new, and its code is obfuscated. **extdrift runs both versions of an extension in a sealed sandbox, records what each one actually does, and scores the difference.** Our rule: *score the change, not the extension.*

What the demo shows

A local web page takes two versions of one extension — the trusted release (N) and the update (N+1) — runs each in an identical, sealed, offline browser, and shows the whole process: the capability the update declares, the two runs happening, the behaviour each one produced with the *new* behaviours marked, a chart of how the counts changed, the sandbox collector's record of anything the update tried to upload, and a plain-English verdict with the indicators behind it. Every run also produces a one-page report.

1,047 behavioural pairs in the dataset	**0.983** model PR-AUC (rules alone: 0.868)	**0.998** PR-AUC, synthetic→real transfer

0 / 21 static detection once code is obfuscated

How it works, in one minute

1. **Sealed sandbox.** Each version runs in headless Chromium with a fresh profile. Every network name resolves to an in-process test site, so **no packet leaves the machine**. Fake collector hosts record anything the extension uploads — that log is ground truth.
2. **Instrumentation.** We inject probes into the extension's own content script and background/service-worker, so we see cookie reads, DOM reads of credential fields, API calls and network uploads as they happen.
3. **Differencing.** A behaviour "key" ignores timestamps and run-to-run noise, so the two runs are comparable. The set of behaviours present in N+1 but not in N is the delta.
4. **Scoring, three tiers.** 12 weighted rules (each citing the paper or incident it comes from) → a trained model (HistGradientBoosting) → an anomaly layer. The verdict reports all of them and whether they agree.

The three kinds of pair you can load (one click each)

Guided demos — a built-in sample extension

A small reader extension in three versions: a benign update, a maliciously updated version, and an identical no-change version. These tell the story cleanly and run in ~35 seconds.

Real published extensions — genuine updates

21 real update pairs downloaded from the Firefox add-on store (uBlock Origin, Dark Reader, Ghostery, Bitwarden…). The tool runs on the real published code. These are benign, and the tool says so — the point is that it works on real extensions, not just our samples.

Real extension + injected attack — the honest malicious case

A real published extension as version N, and the *same* extension with one documented attack behaviour added as version N+1 (cookie-jar theft and exfiltration, modelled on the Cyberhaven incident). This is the most faithful malicious example available — see "Why we inject the attack" below.

Step-by-step: how to run it in the room

Start once, before the review: `python demo/server.py` (opens `http://127.0.0.1:8765`). Then, for each act, click a pair and press **Analyse this update**. Narrate while the stages run; each analysis takes about 35 seconds.

ACT	LOAD THIS	EXPECTED VERDICT	WHAT TO POINT AT
1	Guided → **Malicious update**	● MALICIOUS	The new rows highlighted in the N+1 table; the collector panel ("data actually left"); the indicators, each citing a paper/incident.
2	Guided → **Benign update**	● BENIGN	Behaviour did change (new rows), but nothing dangerous — no exfil, no new capability. The tool does not cry wolf.
3	Guided → **No change at all**	● NO CHANGE	Identical behaviour both runs. This is the control: it proves the sandbox is deterministic and the tool has no false positives on an unchanged update.
4	Real → e.g. **uBlock Origin**	● BENIGN	This is a real published extension, real code. The tool runs on it and clears it.
5	Attack → e.g. **Old Reddit Redirect**	● MALICIOUS	A real extension + one injected attack. The update now reads the whole cookie jar and uploads it to a host it never contacted before; the collector proves the upload landed.

For the live malicious run, prefer an MV3 extension — **Old Reddit Redirect** or **FoxyProxy**: we validated both as MALICIOUS with the exfiltration proven by the collector. (MV3 service workers run most reliably in the sandbox; some larger MV2 extensions land on SUSPICIOUS.) After any run, click **View the full report** or **Print / Save as PDF**.

Pointers to make to the professor

- **Static review misses this; behaviour catches it.** Our reproduction of prior static work detects a malicious change on clean code but **0 of 21** once the same code is obfuscated. Behaviour is unchanged by obfuscation — the extension still dumps the cookie jar.
- **Every indicator is grounded.** Each rule cites the paper or real incident it comes from (ExtPrivA S&P'23, Hulk USENIX'14, the Cyberhaven incident…). It is not a black box.
- **Ground truth, not inference.** The sandbox is sealed and offline; the fake collector's log is what *actually* left the browser, so "data exfiltrated" is observed, not guessed.
- **It generalises.** Trained on synthetic data, it holds up on held-out **real** data (PR-AUC 0.998). The model is not memorising our generator.
- **We fixed a subtle correctness bug.** A service-worker report race once made an unchanged extension look different between two runs; we found and fixed it, and five repeat runs are now byte-identical. For a tool whose whole job is differencing, that matters.

The honest data story (be ready for this question)

"Is this real malware?" — Real *benign* update pairs are real: 21 published extensions, run live. Real *malicious update pairs* are not publicly available anywhere: when an extension is caught it is delisted, and the store keeps no archive of its prior versions (our own probe recovered a predecessor for 0 of 15 flagged IDs). No research group has this data.

Why we inject the attack. So we take a *real* extension and add a behaviour modelled on a *documented real incident* (Cyberhaven, Dec 2024: cookie/credential theft). This is a standard, defensible methodology: it gives a labelled malicious half whose behaviour is grounded in a real attack, and — crucially — a differencer needs labelled behavioural *deltas*, not real malicious pairs, to train. The transfer result above is the evidence it generalises.

What is built already

Package unpacking (signed CRX3, XPI, ZIP) · sealed Chromium and Firefox capture engines · instrumentation for MV2 and MV3 · 26 behavioural + 27 static-code features · the behavioural differencer · a 12-rule scorer, a trained model, and an anomaly layer · a synthetic generator and a weaponiser · a Docker-hardened pipeline · statistical rigor (confidence intervals, significance tests, a locked held-out test) · a reproduction of prior work · an adware sub-study · and this demonstration front end. Remaining work is mainly scaling the real benign corpus, a graph-based model, and the written report.
